Git: manual commit and push

11 Oct 2026 · repo **pokedex** → GitHub **alphanatic/pokeggdex** · commit **44e0d5c**

local cmds | concepts | gotchas | best practice | paths / tables | config | NEW new repo only | EXISTING existing repo only | EVERY every push

1 · The big picture

Three places a file can be

Working folder	Index (staging)	HEAD	origin
your files on disk	`.git/index` — the next commit	your last commit	GitHub's copy

`git add` moves folder → index · `git commit` seals index → HEAD · `git push` sends HEAD → origin.

`git status` compares them: folder ≠ index = red "not staged", index ≠ HEAD = green "to be committed".

2 · The push routine

EVERY Look before you touch

```
cd ~/Projects/pokedex
git status            # what changed, what's staged
git remote -v         # which GitHub repo this folder talks to
git log --oneline -5  # last 5 commits, one line each
```

NEW Brand-new repo only

```
git init -b main             # start
git remote add origin <url>  # link GitHub
git push -u origin main      # 1st push
```

Check `.gitignore` first: `node_modules/`, `dist/`.

EXISTING Your case today

```
git pull --rebase  # sync first
git push           # then push
```

`git remote -v` already showed `origin` → skip the NEW steps.

EVERY Stage, commit, push

```
git add -A                    # stage everything: modified, deleted, new
git status                    # check: all green
git commit -m "Title" -m "Description."   # 1st -m = title, 2nd = body
git push                      # HTTPS remote: username + access token
git status                    # expect "up to date with 'origin/main'"
```

3 · Reading git status

Section	Colour	Meaning	Move it with
On branch / up to date	—	Where you stand vs GitHub	`git pull / push`
Changes to be committed	green	Staged, goes in next commit	`git restore --staged`
Changes not staged	red	Tracked, edited, left out	`git add`
Untracked files	red	Git has never seen them	`git add / .gitignore`

Starship prompt shorthand: `»` renamed · `!` modified · `?` untracked · `✘` deleted · `+` staged · `↑` ahead of GitHub (push me).

4 · Commit messages

Shape of a good message

- **Title:** one short line, sentence case, what changed, no full stop. Shown by `git log --oneline`.
- **Body:** after a blank line — the what and why. Each extra `-m` = one more paragraph (no limit).
- **One change per commit** is best practice; `git add -A` for "everything at once" is fine when it all belongs together.

```
git commit -m "Lore page board, cabin and Pokédex updates" -m "Lore content now sits on a padded board, …"   # today's commit
```

Trailers: `Key: value` lines in the last paragraph

`Co-Authored-By:`	Someone helped — GitHub shows both avatars
`Signed-off-by:`	"I wrote it and may submit it" (Linux kernel style)
`Reviewed-by / Tested-by`	Credit checkers and testers
`Closes: #12`	Auto-closes GitHub issue 12

```
git commit -s -m "Title"                        # -s adds Signed-off-by
git commit -m "Title" --trailer "Closes: #12"    # any trailer, no extra -m
git log -1 --format='%(trailers)'                # show only trailers
```

5 · Inside .git/index (with xxd)

The staging area is a real binary file

A list of **path → content hash**, plus each file's stat info (size, timestamps, inode) so `git status` can skip re-reading unchanged files. File contents live in `.git/objects`, not here.

```
xxd .git/index | head -8              # first 128 bytes as hex
git ls-files --stage .gitignore       # Git's decoded view of one entry
git show :src/pokedex/pokedex.css     # ":" = the staged version of a file
```

Offset	Bytes (your repo)	Meaning
0x00	4449 5243	"DIRC" — I'm a Git index
0x04	0000 0002	Format version 2
0x08	0000 0059	89 tracked files
0x0c	6ac8 e401 …	Entry 1 stat data: times, inode, mode 100644, uid 1000, size 18
0x34	f062 35c4 … 1b08	Content hash (SHA-1) — the actual snapshot
0x48	000a	Path is 10 characters
0x4a	2e67 6974 …	".gitignore" (first: entries sorted, "." before letters)
0x54	0000 … 0000	1–8 NUL bytes, pads entry to a multiple of 8

Reading an xxd row: **left** = byte position · **middle** = bytes in hex · **right** = same bytes as text (dots = unprintable).

6 · Gotchas from today

- `git rmove -v` → typo; Git suggests the closest real command (`remote`).
- `xxd: command not found` → zsh's helper searched pacman's file database. Three packages provide it; picked **tinyxxd** (smallest). Default (1) was gvim — don't just hit Enter.
- `database file for 'chaotic-aur' does not exist` → file lists never synced: `sudo pacman -Fy`.
- Red **404**s while installing → mirrors moved to a newer version than your package list. Fix with `sudo pacman -Syu`; never `-Sy` alone (partial upgrade).
- `git commit -am` skips untracked files — would have missed the new `pokedex.css`. Use `git add -A`.
- Unstaging a rename splits it into `deleted` + `untracked`; re-adding both rejoins it as `renamed (98%)`.
- HTTPS remote: GitHub wants a **personal access token**, not your password.

7 · Open items

```
git push                                    # send 44e0d5c (prompt shows ↑ until done)
pass insert github/token-pokeggdex          # keep the token in pass
git remote set-url origin git@github.com:alphanatic/pokeggdex.git   # later: switch to SSH
```